Intelligent Conversational AI Chatbot

Automated Customer Support for Billing Disputes & Offer Recommendations

NLP · Deep-Learning NLU · Hub-and-Spoke Agents · Reinforcement Learning

MCA Final Project — Proof of Concept · 2026

The Problem

- Telecom support is dominated by two repetitive intents: **“why is my bill wrong?”** and **“is there a better deal?”**
- Human agents are costly, inconsistent, and hold times hurt satisfaction.
- These flows are rule-heavy and latency-sensitive — ideal for AI automation.

Goal: an AI agent that understands free-text queries, acts on the real account, resolves the issue, and escalates hard cases — without fabricating any charge or offer.

Objectives

1. Automated, real-time conversational support using NLP.
2. Improve response quality & efficiency (↑ satisfaction, ↓ response time).
3. Deep-learning NLU + NLG for context-aware, human-like interactions.
4. Reinforcement learning for continuous improvement from feedback.
5. Evaluate: accuracy, intent precision, resolution rate, latency, satisfaction.

What the Chatbot Does

🧾 **Understand your bill**
Explains changes line-by-line.

⚖️ **Dispute a charge**
Shows evidence; resolves via credit, waiver or ticket.

🎁 **Get offers**
Personalized, RL-optimized recommendations.

- **Identity validation** before any account data is shown.
- **Hub-and-spoke** handoff from front-line agent to retention specialist.
- **Grounded NLG** — optional local LLM only rephrases, never invents facts.

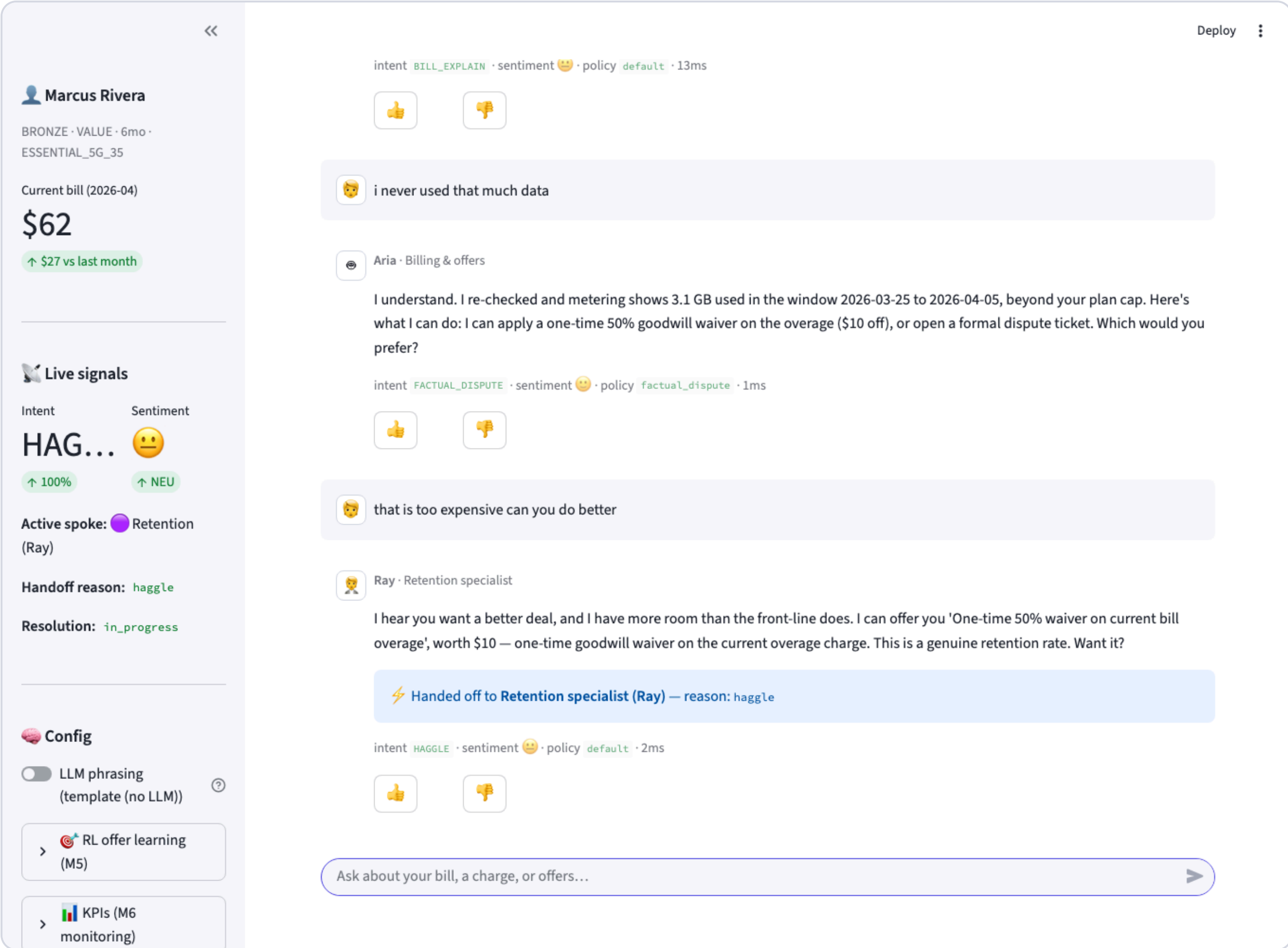
Architecture — “Thick Brain, Thin UI”

Per-turn pipeline: **NLU** → **Policy** → **Hub routing** → **Spoke flow** → **Recommendation (+RL)** → **NLG**

Layer	Does
Presentation (Streamlit)	Auth, chat, live signals, KPIs
Brain (Coordinator + spokes)	Understand, route, decide, act
Data & config (JSON, .env)	Accounts, offers, models, provider

Two AI spokes — **Aria** (billing & offers) and **Ray** (retention) — never talk directly; only the hub moves control.

Hub-and-Spoke in Action



NLU — Deep-Learning Understanding (M3)

- **Intent** — 14 classes, TF-IDF → **MLP neural network**, keyword fallback.
- **Sentiment** — POS/NEU/NEG MLP → signed score + intensity.
- **Entities** — regex slot-filling: charge ids, amounts, dispute topic, yes/no.
- Fused into a single **TurnFrame** consumed by the coordinator.

Every turn shows live intent, sentiment, and policy signals in the UI sidebar.

NLG — Grounded & Pluggable (M4)

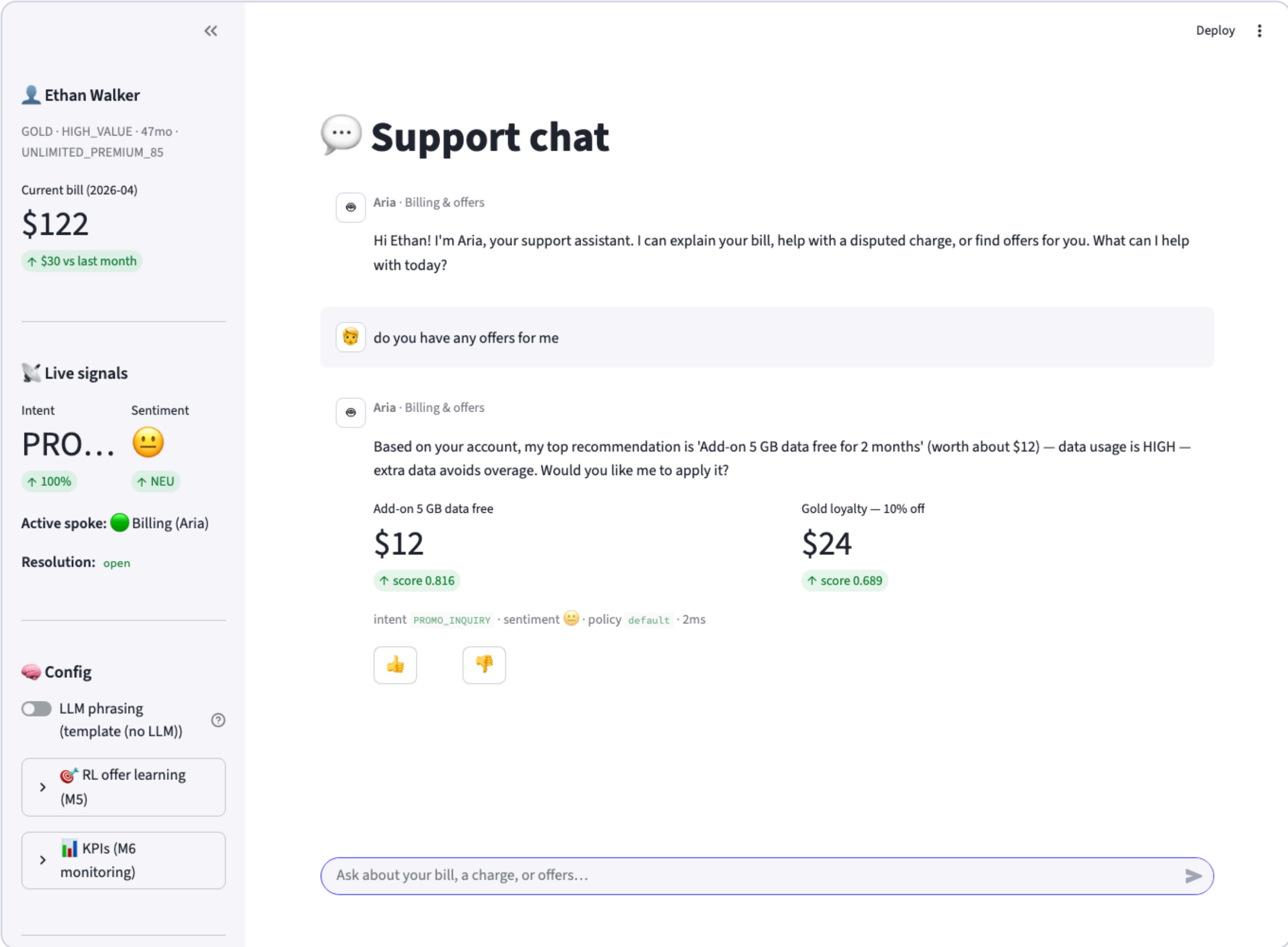
- Flows emit a deterministic **template + grounded facts**.
- Optional **local LLM (Ollama · llama3.2)** rephrases in the persona voice.
- Provider set in `.env`: ollama template openai anthropic
- LLM is told **not to invent** numbers → no hallucinated bills/offers.
- Any LLM failure → automatic template fallback (demo never breaks).

Reinforcement Learning (M5)

- Epsilon-greedy **bandit**: each offer is an arm with shows/accepts.
- Reward: **accept = 1, decline = 0**; value Q = accepts / shows.
- Ranking blends learned value: `score = 0.7·base + 0.3·Q`.
- Feedback from accept/decline and 👍/👎; **policy persists** across sessions.

Effect: offers customers actually accept float to the top over time — continuous improvement from feedback loops.

Offer Recommendation (UC-02)



Results & Evaluation

Metric	Value	Source
Sentiment accuracy (held-out)	~83%	MLP, stratified split
Intent accuracy (held-out)	~62%*	MLP, small seed corpus
Resolution rate	100%	Test scenarios
Avg per-turn latency	~2 ms	Template mode
CSAT proxy	👍/👎 feedback	Session logs

*Limited by the small PoC seed corpus; runtime trains on full data + keyword fallback. Reproducible via `python evaluate.py`.

Tech Stack

UI — Streamlit chat

NLU — scikit-learn MLP + TF-IDF

NLG — templates + Ollama LLM

RL — epsilon-greedy bandit

Data — synthetic JSON

Config — .env provider switch

Runs entirely on a laptop — one command: `./run.sh`

Conclusion & Future Scope

Delivered: an end-to-end conversational agent that understands queries, resolves billing disputes, recommends & applies offers, escalates via hub-and-spoke, learns from feedback, and is fully observable — meeting all five objectives.

Future scope:

- Mine NLU training data from real chat logs; add a transformer NLU.
- Wire live CRM / billing systems in place of synthetic adapters.
- Contextual RL policy over customer features; supervisor approval workflow.
- Proactive validation of promised actions to cut repeat contacts.

Project Milestones & Outcomes

Milestone	Outcome
M1 Data Collection	Synthetic dataset for 3 customers exercising every use case
M2 Data Preparation	Labelled 14-intent + 3-class sentiment corpus + preprocessing
M3 NLU Model	Trained MLP intent + sentiment + entities (~83% sentiment)
M4 NLG Module	Grounded templates + local Ollama LLM, hallucination-free
M5 Reinforcement Learning	Epsilon-greedy bandit, persisted, improves offer ranking
M6 Deployment & Monitoring	One-command Streamlit app, live signals + KPIs

Future State — Roadmap

Stronger AI
Transformer-based NLU; corpus mined from real chat logs; retrieval-augmented answers for open FAQs.

Smarter learning
Contextual RL policy over customer features; real supervisor-approval workflow.

Live integration
Replace synthetic adapters with production CRM, billing & ticketing connectors.

Scale & reach
Cloud deployment behind an autoscaling endpoint; responsive mobile experience; voice channel.

Future Use Cases

1 · Proactive offers on login (SOP-driven)
Evaluate the customer's profile against Standard-Operating-Procedure rules at login and surface the single most relevant eligible offer unprompted — reactive → proactive relationship manager.

2 · Post-application validation agents
After a credit / waiver / offer is applied, a background agent re-reads the authoritative state to confirm fulfilment and auto-remediates discrepancies — closes the promise-to-fulfilment loop.

3 · Call reduction via proactive network-telemetry fixes
Ingest throttling / outage / coverage signals and pre-emptively credit or re-plan affected customers before they contact support — shifts the operator from reactive to preventive.

Thank You

Demo: `./run.sh` → http://localhost:8501

Repository & docs: README.md · architecture/ · docs/demo/demo.mp4